

NEHA SHARMA U24AI050 LAB 13

10 April 2026 16:24

1. Write a program to simulate segmentation using a static segment table and perform address translation.

```
#include <stdio.h>

struct Segment {
    int base;
    int limit;
};

int main() {
    struct Segment segTable[3] = {
        {1000, 400}, // Segment 0
        {2000, 300}, // Segment 1
        {3000, 500}  // Segment 2
    };

    int seg_no, offset;

    printf("Enter Segment Number (0-2): ");
    scanf("%d", &seg_no);

    printf("Enter Offset: ");
    scanf("%d", &offset);

    if (seg_no < 0 || seg_no > 2) {
        printf("Invalid Segment Number\n");
        return 0;
    }

    if (offset >= segTable[seg_no].limit) {
        printf("Offset exceeds limit! Segmentation Fault\n");
    } else {
        int physical_address = segTable[seg_no].base + offset;
```

```

    printf("Physical Address = %d\n", physical_address);
}

return 0;
}

```

```

ubuntu@ubuntu: ~
ubuntu@ubuntu:~$ gcc 13_1.c -o output
ubuntu@ubuntu:~$ ./output
Enter Segment Number (0-2): 1
Enter Offset: 200
Physical Address = 2200
ubuntu@ubuntu:~$ ./output
Enter Segment Number (0-2): 4
Enter Offset: 400
Invalid Segment Number
ubuntu@ubuntu:~$ ./output
Enter Segment Number (0-2): 2
Enter Offset: 250
Physical Address = 3250
ubuntu@ubuntu:~$
ubuntu@ubuntu:~$
549 MB Volume
ubuntu@ubuntu:~$
ubuntu@ubuntu:~$
ubuntu@ubuntu:~$
ubuntu@ubuntu:~$
ubuntu@ubuntu:~$
ubuntu@ubuntu:~$

```

2. Create a simulation that takes a logical address (segment number + offset) and translates it to a physical address using a segment table.

```
#include <stdio.h>
```

```

struct Segment {
    int base;
    int limit;
};

```

```
int main() {
```

```

int n;
printf("Enter number of segments: ");
scanf("%d", &n);

struct Segment segTable[n];

for (int i = 0; i < n; i++) {
    printf("Enter base and limit for segment %d: ", i);
    scanf("%d %d", &segTable[i].base, &segTable[i].limit);
}

int seg_no, offset;
printf("Enter Segment Number: ");
scanf("%d", &seg_no);

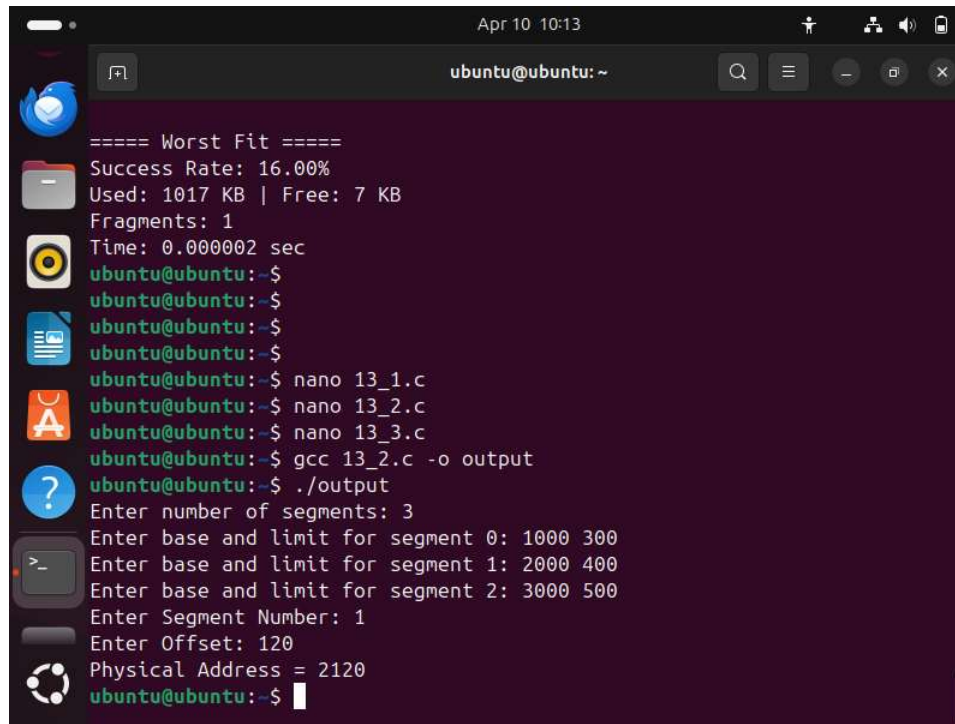
printf("Enter Offset: ");
scanf("%d", &offset);

if (seg_no >= n || seg_no < 0) {
    printf("Invalid Segment Number\n");
    return 0;
}

if (offset >= segTable[seg_no].limit) {
    printf("Segmentation Fault!\n");
} else {
    int physical_address = segTable[seg_no].base + offset;
    printf("Physical Address = %d\n", physical_address);
}

return 0;
}

```



A terminal window on Ubuntu showing the output of a memory allocation program. The output includes statistics like 'Worst Fit', 'Success Rate: 16.00%', and 'Used: 1017 KB | Free: 7 KB'. It also shows the execution time and a series of user prompts for segment information, which have been answered with values like 3 segments, base/limit pairs (1000 300, 2000 400, 3000 500), segment number 1, and offset 120, resulting in a physical address of 2120.

```
==== Worst Fit ====
Success Rate: 16.00%
Used: 1017 KB | Free: 7 KB
Fragments: 1
Time: 0.000002 sec
ubuntu@ubuntu:~$
ubuntu@ubuntu:~$
ubuntu@ubuntu:~$
ubuntu@ubuntu:~$
ubuntu@ubuntu:~$ nano 13_1.c
ubuntu@ubuntu:~$ nano 13_2.c
ubuntu@ubuntu:~$ nano 13_3.c
ubuntu@ubuntu:~$ gcc 13_2.c -o output
ubuntu@ubuntu:~$ ./output
Enter number of segments: 3
Enter base and limit for segment 0: 1000 300
Enter base and limit for segment 1: 2000 400
Enter base and limit for segment 2: 3000 500
Enter Segment Number: 1
Enter Offset: 120
Physical Address = 2120
ubuntu@ubuntu:~$
```

3. Implement a model where each process is divided into segments (code, data, stack), and manage memory allocation and deallocation for each.

```
#include <stdio.h>
```

```
struct Segment {
    int base;
    int limit;
};
```

```
int main() {
    struct Segment code, data, stack;
    int memory = 10000;
    int current_base = 0;
```

```
// Allocate Code Segment
printf("Enter size of Code segment: ");
scanf("%d", &code.limit);
```

```

code.base = current_base;
current_base += code.limit;

// Allocate Data Segment
printf("Enter size of Data segment: ");
scanf("%d", &data.limit);
data.base = current_base;
current_base += data.limit;

// Allocate Stack Segment
printf("Enter size of Stack segment: ");
scanf("%d", &stack.limit);
stack.base = current_base;
current_base += stack.limit;

if (current_base > memory) {
    printf("Memory Overflow!\n");
    return 0;
}

printf("\nSegment Allocation:\n");
printf("Code -> Base: %d, Limit: %d\n", code.base, code.limit);
printf("Data -> Base: %d, Limit: %d\n", data.base, data.limit);
printf("Stack -> Base: %d, Limit: %d\n", stack.base, stack.limit);

// Deallocation simulation
printf("\nDeallocating Data Segment...\n");
data.base = -1;
data.limit = 0;

printf("Data segment freed!\n");

return 0;
}

```

```
Apr 10 10:45
ubuntu@ubuntu: ~
ubuntu@ubuntu:~$ gcc 13_3.c -o output
Rhythmbox ubuntu:~$ ./output
Enter size of Code segment: 2000
Enter size of Data segment: 3000
Enter size of Stack segment: 1000

Segment Allocation:
Code -> Base: 0, Limit: 2000
Data -> Base: 2000, Limit: 3000
Stack -> Base: 5000, Limit: 1000

Deallocating Data Segment...
Data segment freed!
ubuntu@ubuntu:~$ 5000
5000: command not found
ubuntu@ubuntu:~$ ./output
Enter size of Code segment: 5000
Enter size of Data segment: 4000
Enter size of Stack segment: 2000
Memory Overflow!
ubuntu@ubuntu:~$
```